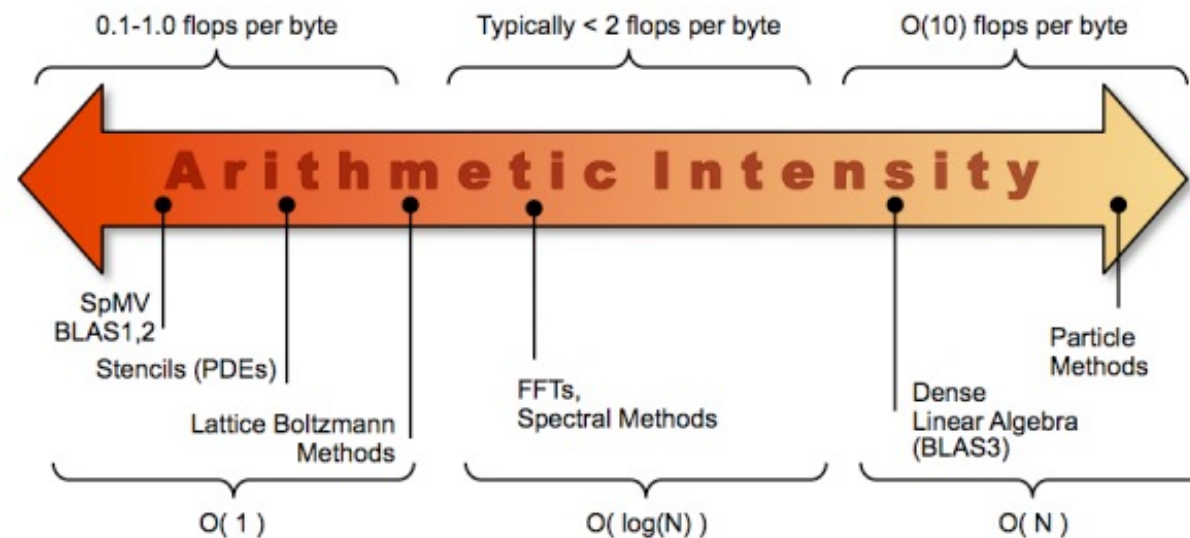


Lock-free Algorithms (& fine-grain //)

Parallel Algorithms

João Luís Sobral (jls@di ...)

April/2026



Data storage

Regular : vector, matrix

Irregular (**pointer based**):

linked-list, tree, graphs

Data accesses

- Regular / **irregular**

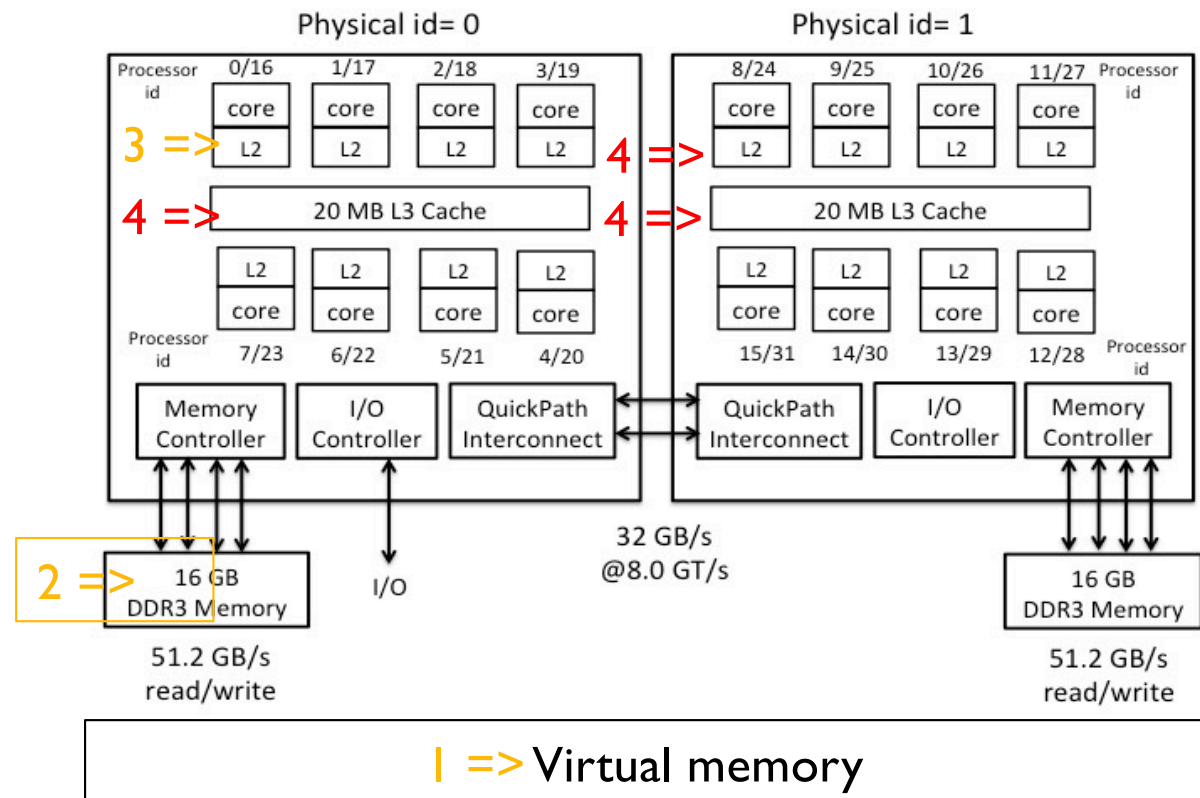
Recap: memory hierarchy (a detailed view)

1 => Data is allocated in **virtual memory**

2 => on Linux: physical memory page(s) are reserved on a memory bank on the “first-touch”

3 => Data is cached on local L3/L2/L1 as threads read the data from the shared memory

4 => Multiple data copies are kept coherent by the hardware

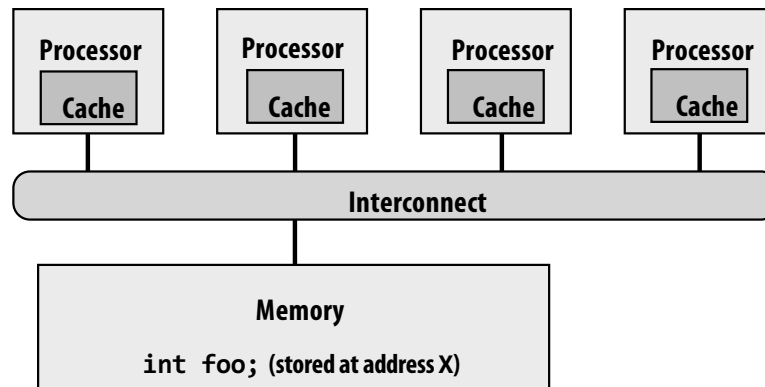


- **Data locality:** the hardware will try to keep the data closer to the PUs (i.e., threads) using it

- Data movement costs?

Recap?: The cache coherence problem

- Modern processors replicate contents of memory in local caches
 - Problem:** processors can observe different values for the same memory location



Is this a mutual exclusion problem?
Can we fix the problem by adding locks to the program?

NO!

This is a problem created by replicating the data stored at address X in local caches

- The chart at right shows the value of variable foo (stored at address X) in main memory and in each processor's cache
 - Assume the initial value stored at address X is 0
 - Assume write-back cache behavior
 - How could we fix this problem?**

Action	P1 \$	P2 \$	P3 \$	P4 \$	mem[X]
					0
P1 load X	0 miss				0
P2 load X	0	0 miss			0
P1 store X	1	0			0
P3 load X	1	0	0 miss		0
P3 store X	1	0	2		0
P2 load X	1	0 hit	2		0
P1 load Y (assume this load causes eviction of X)		0	2		1

- **Intuitive behavior for memory system:** reading value at address X should return the last value written to address X by any processor.
- Memory coherence problem exists because there is both global storage (main memory) and per-processor local storage (processor caches) implementing the abstraction of a single shared address space.

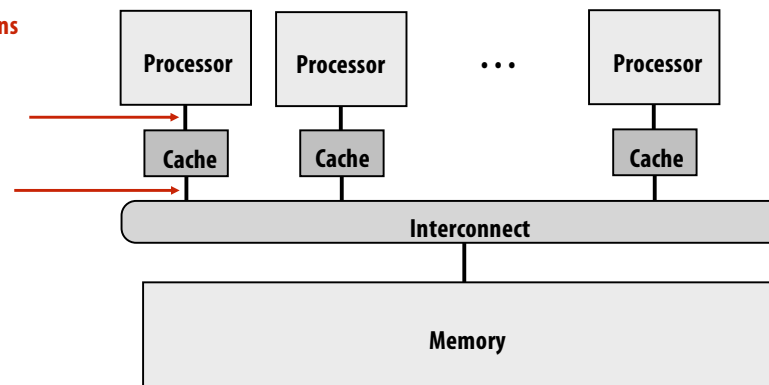
Implementing coherence

- Software-based solutions (coarse grain: VM page)
 - OS uses page-fault mechanism to propagate writes
 - Can be used to implement memory coherence over clusters of workstations
 - Big performance problem: false sharing (discussed later)
- Hardware-based solutions (fine grain: cache line)
 - Directory-based coherence implementations
 - “Snooping”-based coherence implementations
 - **Main idea:** all coherence-related activity is broadcast to all processors in the system (more specifically: to the processor’s cache controllers)
 - Cache controllers monitor (“they snoop”) memory operations, and follow cache coherence protocol to maintain memory coherence

Notice: now cache controller must respond to actions from “both ends”:

1. LD/ST requests from its local processor

2. Coherence-related activity broadcast over the chip’s interconnect



Communication time is a key parallel overhead

- $$AMAT_{\text{Multiprocessor}} > AMAT_{\text{Uniprocessor}}$$

Uniprocessor

Register
L1 Cache
L2 Cache
Main Memory
Remote

Multiprocessor

Register, less register allocation
L1 Cache, higher miss rate
L2 Cache, higher miss rate
Main, can "miss" in NUMA
Remote, extra long delays

Core i7 Xeon 5500 Series Data Source Latency (approx.)	
L1 hit,	~4 cycles
L2 hit,	~10 cycles
L3 hit, line unshared	~40 cycles
L3 hit, shared line in another core	~65 cycles
L3 hit, modified in another core	~75 cycles remote
Local DRAM	~30 ns (~120 cycles)
Remote DRAM	~100 ns (~400 cycles)

Width indicates frequency of access

Stanford CS149, Fall

5

Implications to the programmer

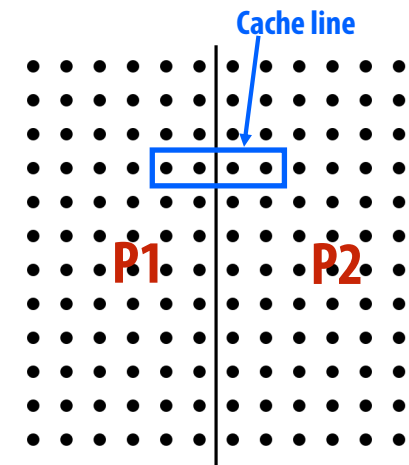
False sharing

Condition where two processors write to different addresses, but addresses map to the same cache line

Cache line “ping-pongs” between caches of writing processors, generating significant amounts of communication due to the coherence protocol

No inherent communication, this is entirely artificial communication (cachelines > 4B)

False sharing can be a factor in when programming for cache-coherent architectures



Models of memory consistency

- Execution of multi-threaded programs in uniprocessors (single core)
 - The processor executes all threads in the program
 - Thread scheduling policy is not specified
 - Operations executed by each thread are performed **in order**
 - Thread synchronization with **atomic operations** (e.g. locks)
 - Operations executed by threads are intermixed
 - **Non-determinism**: the result depends on the order in which threads are scheduled, e.g.:

Thread 1	Thread 2
..... x := 1; x := 2;	 print(x);

Models of memory consistency

- Execution of multi-threaded programs in **multiprocessors**
 - Each processor executes a thread (considering the number of threads equal to the number of processors)
 - Operations executed by each thread are performed **in order**
 - Each processor can access to global memory to perform load/store/atomic operations
 - No caching of global data (for now!)
 - Possible results are equal to single processor.
 - More realistic considerations:
 - Caching of global data to improve efficiency
 - Requires a protocol for cache cohesion
 - Out-of-order instruction execution
 - Can change the program semantic => a memory consistency model is required!

Models of memory consistency

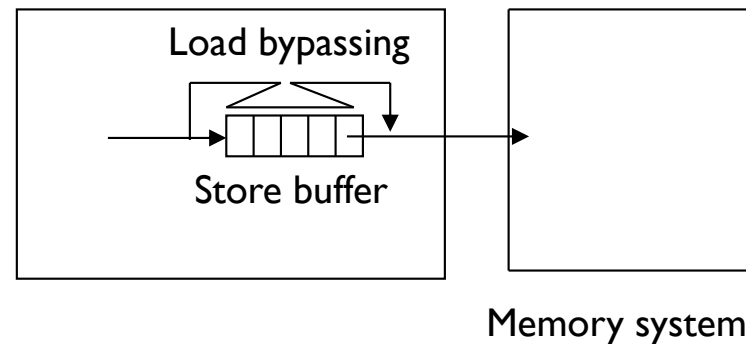
- **Detail:** out-of-order instruction execution
 - Processors reorder instructions to improve performance
 - Reorder of instructions should respect program dependencies
 - Data– e.g., loads/stores should be executed by the required order
 - Control dependencies
 - Instruction reorder can be performed by the compiler or by the processor
 - Allowed reorders:
 - Stores in different locations

store v1, data		store b1, flag
store b1, flag	↔	store v1, data
 - Loads from different locations

load flag, r1		load data, r2
load data, r2	↔	load flag, r1
 - Loads and stores from/in different locations (details in next slide)

Models of memory consistency

- **Detail:** out-of-order instruction execution (load/stores)
 - Example of hardware instruction reorder



- The store buffer saves store operations to be sent to memory
- loads have priority through stores to keep the processor busy
 - They can bypass stores in the buffer
- loads verify if there is a store in the same target address
- Result: load and stores are not performed in the program order

Models of memory consistency

- Out-or-order execution in multiprocessors
 - Canonical model (ideal!)
 - Operations performed by a given processor are done in-order
 - Operations performed by different processors are intermixed
 - **More realistic model:**
 - If a processor reorders his instruction flow, will be the same result accomplished?

Initially $A = \text{Flag} = 0$

P1

```
A = 23;  
Flag = 1;
```

P2

```
while (Flag != 1) {}  
... = A;
```

- P1 writes data in A and signals it to P2 by changing the Flag value
- P2 waits until Flag is active, and then reads A
- What happens if P1 exchanges the order of stores?

Models of memory consistency

- Out-of-order execution in multiprocessors (cont.)
 - Example II

Flag1 = Flag2 = 0

P1

Flag1 = 1;
If (Flag2 == 0)
critical section

P2

Flag2 = 1;
If (Flag1 == 0)
critical section

One possible order:

P1

Write Flag1, 1
Read Flag2 //get 0

P2

Write Flag2, 1
Read Flag1 // get 1? => only if load and stores are
performed in order

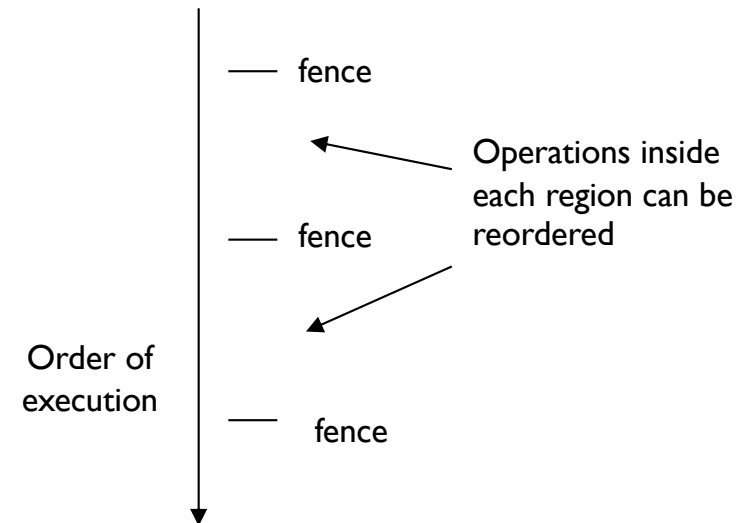
Models of memory consistency

- Conclusion

- Uniprocessors can reorder instructions to improve performance, respecting local dependencies
- This is not valid for multiprocessors sharing memory
 - A parallel program can deliver contra-intuitive results
- What limitations can be imposed to instruction reorder such that:
 - programming can be intuitive
 - do not lose uniprocessor performance
- Solution: memory consistency model supported by the processor:
 - Sequential consistency model (the simplest):
 - Processor can not reorder load/stores in global memory
 - Reduces uniprocessor performance to an unacceptable level

Models of memory consistency

- Relaxed consistency models – weak consistency
 - Programmers specify execution points where operations should be ordered
 - “Local coarse-grain clock”
 - “fence” instruction
 - Operations before “fence” should complete before the “fence” execution
 - Operations after the “fence” must wait until the “fence” terminates.
 - “fences” are executed in order
 - “fence” implementation
 - Counter incremented at each fence execution
 - Example:
 - SYNC instruction in PowerPC
 - Flush in OpenMP
 - Lock/unlock synchronization
 - **Important:** the fence is local to each PU



Models of memory consistency

- Relaxed consistency models – weak consistency
 - Example I - Revised

Initially A = Flag = 0

P1

```
A = 23;  
flush  
Flag = 1;
```

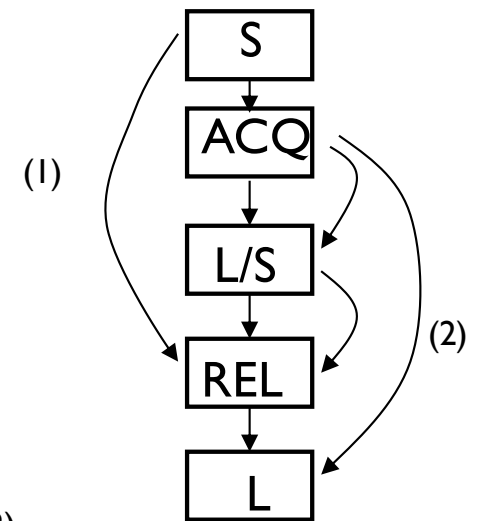
P2

```
while (Flag != 1) {;  
... = A;
```

- P1 writes data to A
- flush waits for the write to A to complete
- P1 activates the Flag
- P2 reads Flag==1 when the write in A has completed, even if operations before the flush are completed out-of-order
- Is a flush required among the two operations in P2?

Models of memory consistency

- Relaxed consistency models – release consistency
 - More relaxed model than weak consistency
 - The weak consistency model does not differentiate reads from writes (the fence instruction implies a read/write of all values that have been changed)
 - The release consistency model updates local values (read from memory) when entering in the critical region and writes the values on the exit of the region
 - Access for synchronization are divided in:
 - acquisition: operations of type lock
 - Should be performed when entering in the critical region
 - Does not wait for the access that precede it (1)
 - release: operations of type unlock
 - Should be executed after all writes
 - Accesses after “release” do not need to wait for the release (2)





Memory consistency vs cohesion

- Memory consistency models
 - Impose restrictions to the reordering of instructions of a single processor
 - It is not related with memory operations of different processors
 - There are many consistency models
 - There is a consensus around weak/release
 - It is easy to write a program whose behaviour is trapped into a specific consistency model
- Memory cohesion
 - Creates the illusion of a single logic localization corresponding to each variable of the program, even if there are multiple localizations for that variable

Models of memory consistency

- OpenMP model
 - Based on weak consistency
 - Activities can work on a temporary view of memory (registers, cache, etc)
 - Reads and writes can be performed on local memory
 - “fence” instructions
 - **#pragma omp flush**
 - Enforces the consistency between the temporary view and the memory (for the list of variables in the flush)
 - Variables changed locally are written into memory
 - Local copies of variables are discarded to enforce reading for the memory on the next local read.
 - Operation ends when all variables have been updated in memory
 - There is no guarantee of ordering among flush if the list of variables is disjoint
 - Memory/local variables (threadprivate, private) are not affected
 - flush is implicit in:
 - Barrier, set_lock, unset_lock, atomic
 - When entering/exiting from parallel, critical and ordered
 - It is not advisable to perform synchronization using variables in memory

Recap: OpenMP Synchronization

- Solving data races with critical VS atomic

```

5  int main(){
6      double result={0};
7
8      #pragma omp parallel for shared(result)
9      for(int i=0; i<1000000;i++) {
10         result+=sin(i);
11     }
12     printf("%f",result);
13 }

```

Data race

%r12 contains a pointer to a shared memory position

```

.L4: vxorpd %xmm0, %xmm0, %xmm0
     vcvtsi2sdl %ebx, %xmm0, %xmm0 ; convert to double
     call sin
     vmovsd 8(%rsp,%r12), %xmm1 ; load result into %xmm1
     vaddsd %xmm0, %xmm1, %xmm1 ; add
     vmovsd %xmm1, 8(%rsp,%r12) ; update result
     ...
     jne .L4

```

- Critical:** coarse-grain and pessimist

```

...
call GOMP_critical_start
vmovsd 8(%rsp,%r12), %xmm1 ; load result into %xmm1
vaddsd %xmm0, %xmm1, %xmm1
vmovsd %xmm1, 8(%rsp,%r12) ; update results
call GOMP_critical_end

```

Mutual exclusion: ensures that only one thread executes the critical region (e.g., blocks the calling thread if another thread is still executing the region)

- Atomic:** fine-grain and optimistic (note: simplified code)

```

.L5: vmovsd (%r12), %xmm1
     vaddsd %xmm0, %xmm1, %xmm1
     ...
     lock cmpxchgq %xmm1, (%r12) ; atomic comp & exchange
     cmpq %rax, %rdx
     jne .L5 ; repeat if update failed

```

Repeat until successful update

Lock instructions

x86 cmpxchg

- Compare and exchange (atomic when used with lock prefix)

`lock cmpxchg dst, src`



lock prefix (designates operation is atomic)



often a memory address

```
if (dst ==  EAX)
    ZF = 1 
    dst = src
else
    ZF = 0
    EAX = dst
```

x86 register

flag register holds result of check

Lock instructions

Test-and-test-and-set lock

```
void Lock(int* lock) {  
    while (1) {  
  
        while (*lock != 0);           // while another processor has the lock...  
                                     // (assume *lock is NOT register allocated)  
  
        if (test_and_set(*lock) == 0) // when lock is released, try to acquire it  
            return;  
    }  
}  
  
void Unlock(int* lock) {  
    *lock = 0;  
}
```

Lock operations

Example: a sorted linked list

What can go wrong if multiple threads operate on the linked list simultaneously?

```
struct Node {
    int value;
    Node* next;
};

struct List {
    Node* head;
};
```

```
void insert(List* list, int value) {

    Node* n = new Node;
    n->value = value;

    // assume case of inserting before head of
    // of list is handled here (to keep slide simple)

    Node* prev = list->head;
    Node* cur = list->head->next;

    while (cur) {
        if (cur->value > value)
            break;

        prev = cur;
        cur = cur->next;
    }

    n->next = cur;
    prev->next = n;
}
```

```
void delete(List* list, int value) {

    // assume case of deleting first node in list
    // is handled here (to keep slide simple)

    Node* prev = list->head;
    Node* cur = list->head->next;

    while (cur) {
        if (cur->value == value) {
            prev->next = cur->next;
            delete cur;
            return;
        }

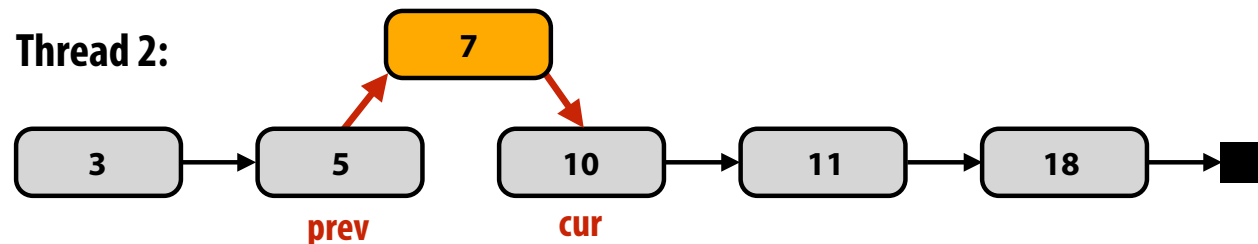
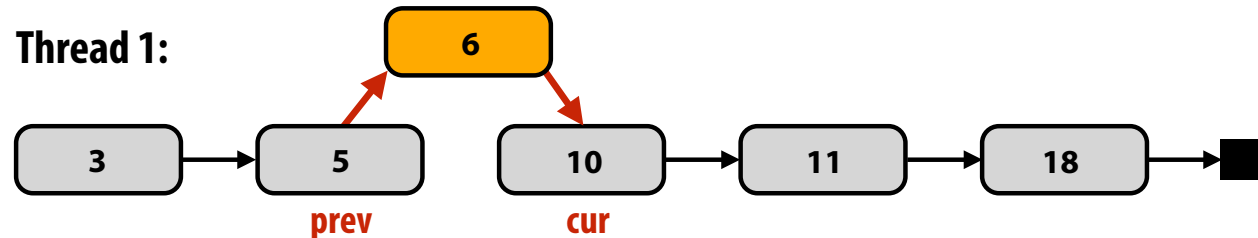
        prev = cur;
        cur = cur->next;
    }
}
```

Lock operations

Example: simultaneous insertion

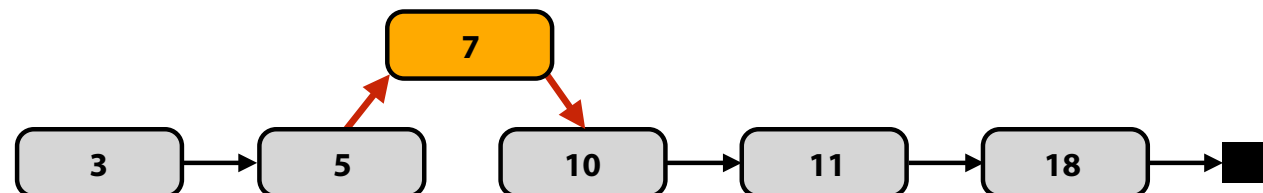
Thread 1 attempts to insert 6

Thread 2 attempts to insert 7



Thread 1 and thread 2 both compute same prev and cur.
Result: one of the insertions gets lost!

Result: (assuming thread 1 updates prev->next before thread 2)



Lock operations

Solution 1: protect the list with a single lock

```
struct Node {
    int value;
    Node* next;
};

struct List {
    Node* head;
    Lock lock; ← Per-list lock
};

void insert(List* list, int value) {
    Node* n = new Node;
    n->value = value;

    lock(list->lock);

    // assume case of inserting before head of
    // of list is handled here (to keep slide simple)

    Node* prev = list->head;
    Node* cur = list->head->next;

    while (cur) {
        if (cur->value > value)
            break;

        prev = cur;
        cur = cur->next;
    }
    n->next = cur;
    prev->next = n;
    unlock(list->lock);
}

void delete(List* list, int value) {
    lock(list->lock);

    // assume case of deleting first element is
    // handled here (to keep slide simple)

    Node* prev = list->head;
    Node* cur = list->head->next;

    while (cur) {
        if (cur->value == value) {
            prev->next = cur->next;
            delete cur;
            unlock(list->lock);
            return;
        }

        prev = cur;
        cur = cur->next;
    }
    unlock(list->lock);
}
```

Lock operations

Solution 2: fine-grained locking

```
struct Node {
    int value;
    Node* next;
    Lock* lock;
};

struct List {
    Node* head;
    Lock* lock;
};

void insert(List* list, int value) {

    Node* n = new Node;
    n->value = value;

    // assume case of insert before head handled
    // here (to keep slide simple)

    Node* prev, *cur;

    lock(list->lock);
    prev = list->head;

    lock(prev->lock);
    unlock(list->lock);

    cur = prev->next;
    if (cur) lock(cur->lock);

    while (cur) {
        if (cur->value > value)
            break;

        Node* old_prev = prev;
        prev = cur;
        cur = cur->next;
        unlock(old_prev->lock);
        if (cur) lock(cur->lock);
    }

    n->next = cur;
    prev->next = n;

    unlock(prev->lock);
    if (cur) unlock(cur->lock);
}
```

Challenge to students: there is way to further improve the implementation of insert(). What is it?

```
void delete(List* list, int value) {

    // assume case of delete head handled here
    // (to keep slide simple)

    Node* prev, *cur;

    lock(list->lock);
    prev = list->head;

    lock(prev->lock);
    unlock(list->lock);

    cur = prev->next;
    if (cur) lock(cur->lock)

    while (cur) {
        if (cur->value == value) {
            prev->next = cur->next;
            unlock(prev->lock);
            unlock(cur->lock);
            delete cur;
            return;
        }

        Node* old_prev = prev;
        prev = cur;
        cur = cur->next;
        unlock(old_prev->lock);
        if (cur) lock(cur->lock);
    }
    unlock(prev->lock);
}
```

Lock-free implementations

Lock-free linked list insertion *

```
struct Node {
    int value;
    Node* next;
};

struct List {
    Node* head;
};

// insert new node after specified node
void insert_after(List* list, Node* after, int value) {

    Node* n = new Node;
    n->value = value;

    // assume case of insert into empty list handled
    // here (keep code on slide simple for class discussion)

    Node* prev = list->head;

    while (prev->next) {
        if (prev == after) {
            while (1) {
                Node* old_next = prev->next;
                n->next = old_next;
                if (compare_and_swap(&prev->next, old_next, n) == old_next)
                    return;
            }
        }
        prev = prev->next;
    }
}
```

Compared to fine-grained locking implementation:

No overhead of taking locks

No per-node storage overhead



Atomic update
of pointer to
next element

* For simplicity, this slide assumes the *only* operation on the list is insert. Delete is more complex.